

Description de la Forme du Projet-Agentic Vectorial Graph RAG-PFA-4eme Année Spécialité Intelligence Artificielle

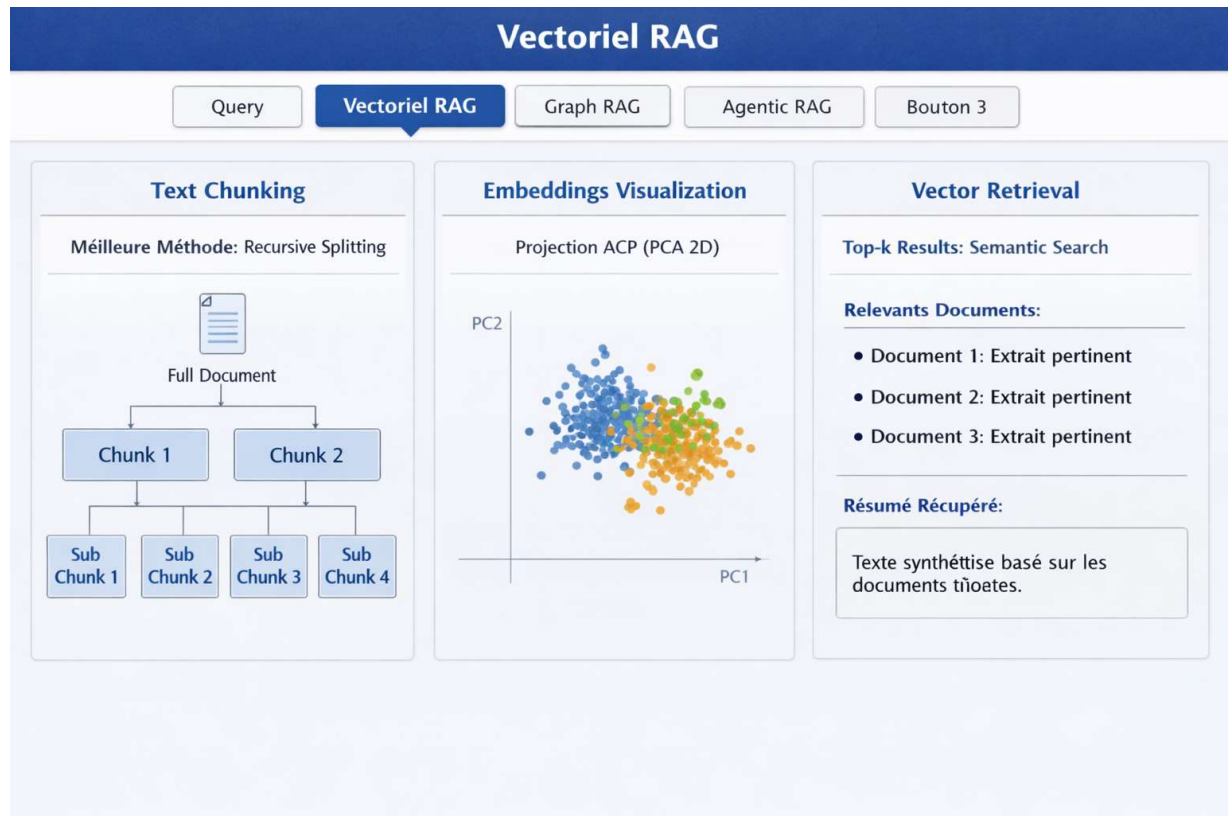
5eme Séance Le 15/5/2026

I-Présentation de la Forme du Projet Interface Maquette et boutons

La Maquette / Interface du projet doit être de la forme Suivante



I-Bouton Vectoriel RAG



Chunking

- **Bouton “Chunking”** → affiche les **7 méthodes** de découpage (par taille fixe, par phrases, par paragraphes, par similarité sémantique, par fenêtre glissante, par modèle récursif, etc.).
- L’interface choisit automatiquement la **meilleure méthode** selon ton corpus (par exemple : *Recursive Splitting*).
- Tu peux visualiser le découpage du texte en blocs hiérarchiques : *Document complet* → *Chunks* → *Sub-Chunks*.

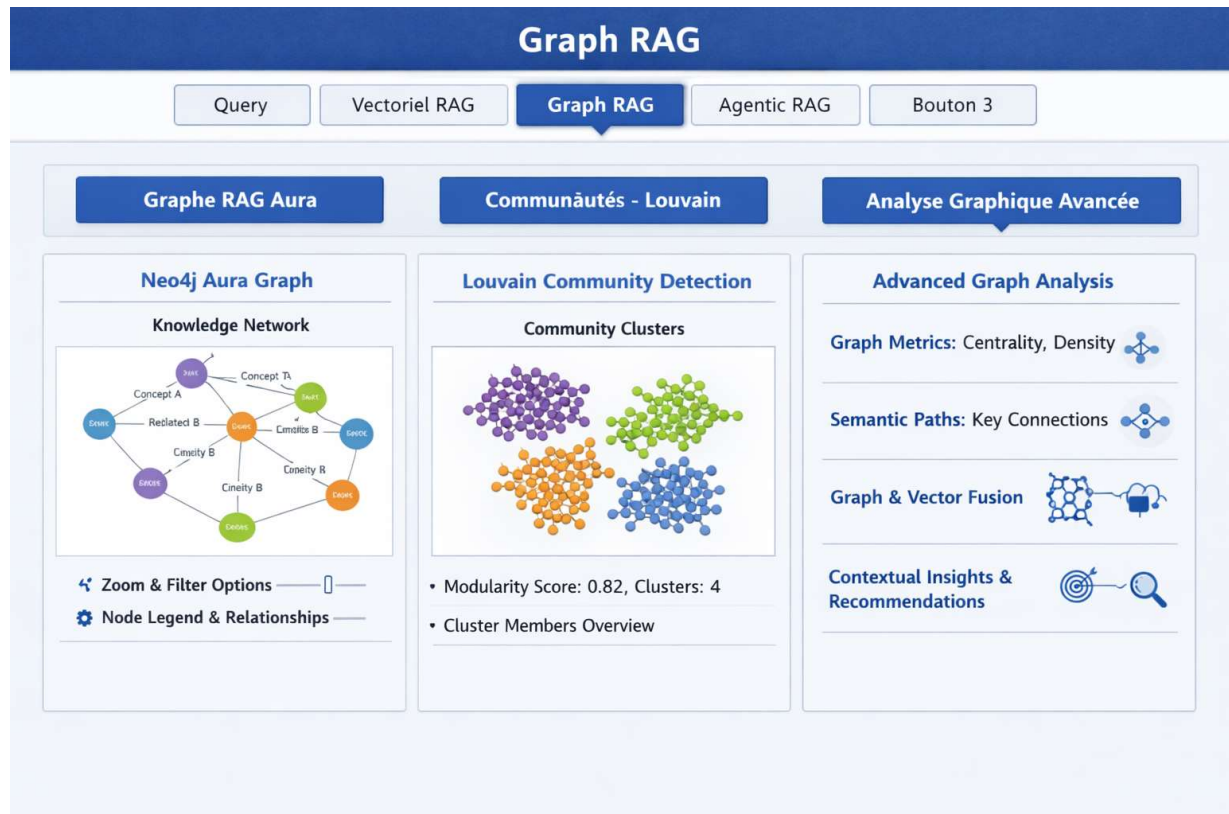
Embeddings FAISS

- **Bouton “Embeddings FAISS”** → affiche la **projection ACP (PCA 2D)** des embeddings pour visualiser la distribution des données.
- Tu vois un **nuage de points colorés** représentant les clusters sémantiques.
- Possibilité d’ajouter un filtre pour afficher les points les plus proches de la requête.

Retrieval Vectoriel RAG

- **Bouton “Retrieval Vectoriel”** → montre la **meilleure méthode de recherche** parmi les 5 vues en TP (ex. : *Top-k Semantic Search*, *Cosine Similarity*, *Hybrid BM25 + Embeddings*, etc.).
- L’interface affiche les **documents les plus pertinents** avec extraits et score de similarité.
- Une zone “Résumé Récupéré” synthétise les résultats pour la génération finale.

2-Bouton Graph-RAG



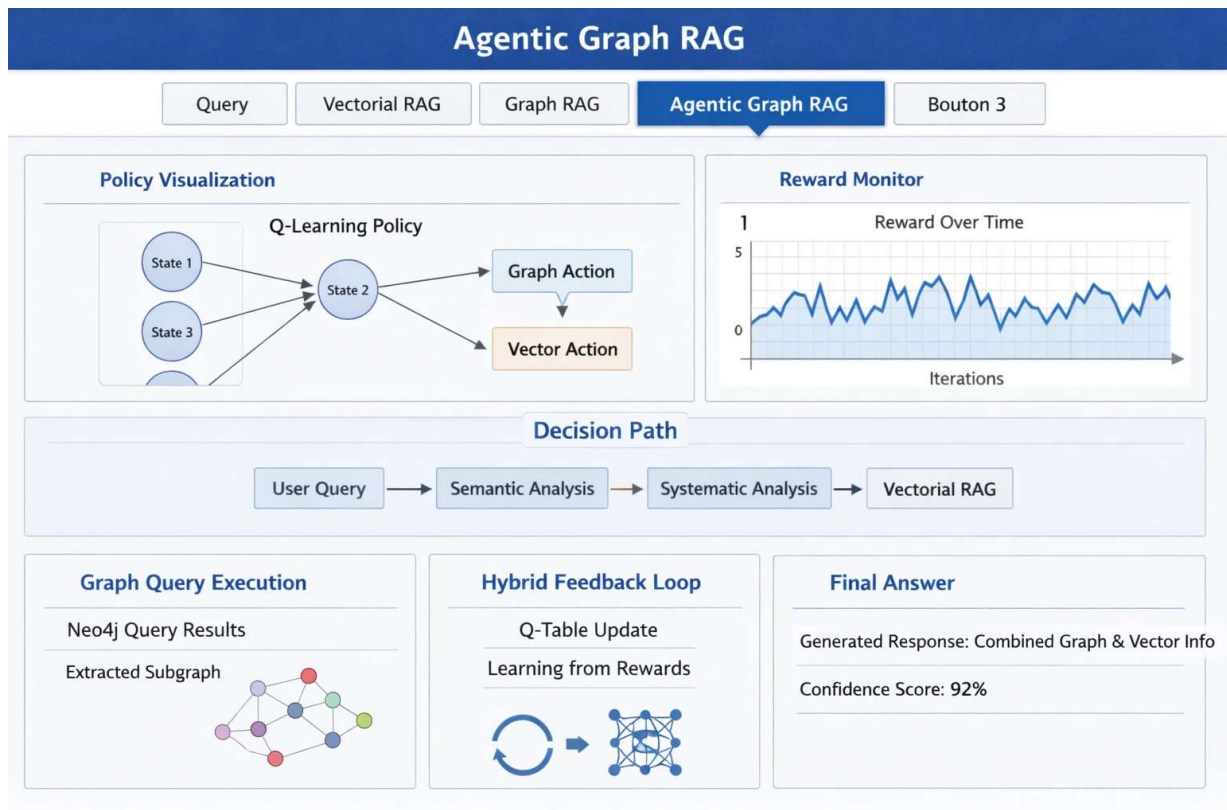
Voici la maquette du bouton **“Graph RAG”** regroupant les trois sous-modules que tu as définis :

Graphe RAG Aura → visualisation Neo4j Aura du réseau de connaissances, avec zoom, filtres et légende des relations.

Communautés – Louvain → carte des clusters colorés, score de modularité et aperçu des membres de chaque communauté.

Analyse Graphique Avancée → métriques (centralité, densité), chemins sémantiques, fusion graphe + vectoriel et recommandations contextuelles.

3-Bouton Agentic-Graph RAG — Interface et logique



Élément	Fonction
Policy Visualization	Affiche la politique Q-Learning : état → action (Graph ou Vectoriel).
Reward Monitor	Montre les récompenses obtenues selon la qualité de la réponse.
Decision Path	Visualisation du chemin choisi : Query → (analyse sémantique ou systématique) → branche Graph ou Vectoriel.
Graph Query Execution	Si la politique choisit Graph RAG, affiche les requêtes Neo4j exécutées et les sous-graphes explorés.
Hybrid Feedback Loop	Montre comment le résultat du Graph RAG est réinjecté dans la mémoire de l'agent pour améliorer la politique Q-Learning.
Final Answer Panel	Résultat final combiné, avec provenance et score de confiance.

Étape 1 : L'agent reçoit la Query et calcule les features (sémantique, systématique, complexité).

Étape 2 : Le module Q-Learning choisit l'action :

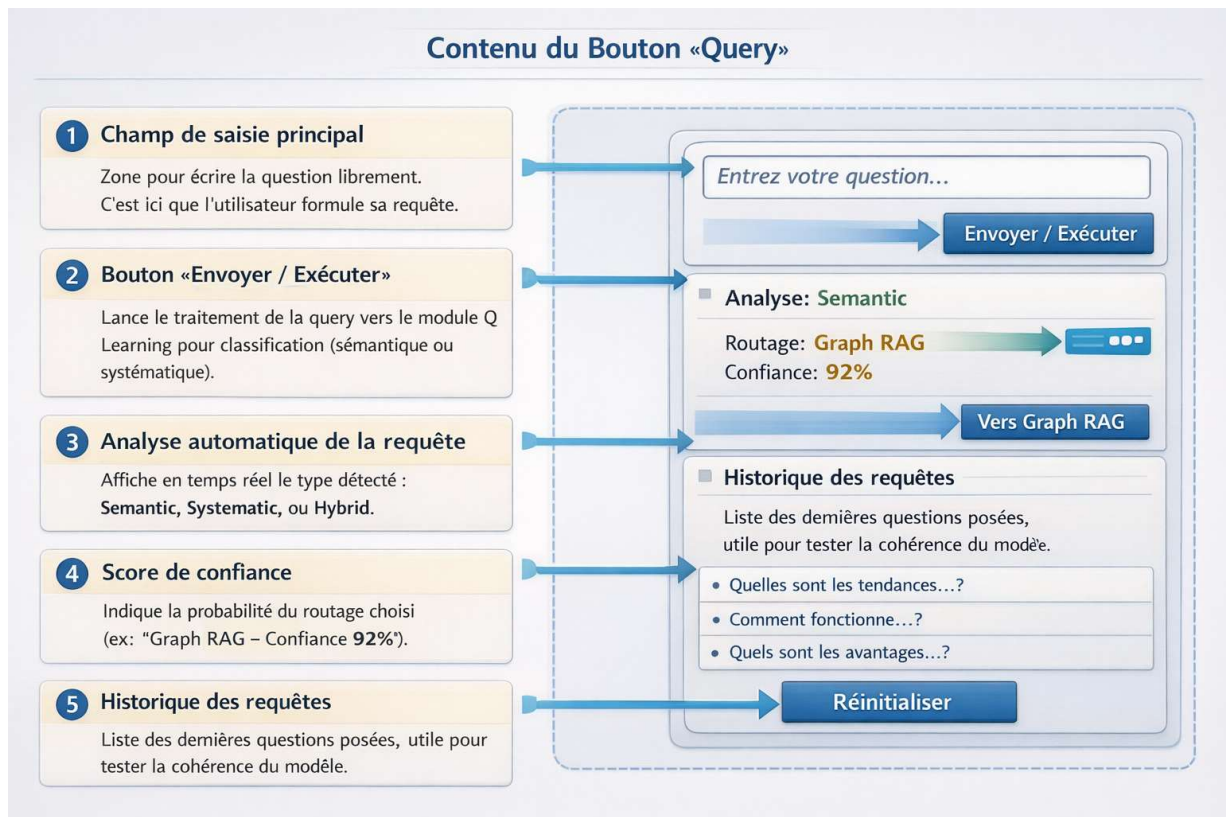
- Action 1 → Graph RAG (si la requête est systématique, relationnelle, ou orientée structure).
- Action 2 → Vectorial RAG (si la requête est sémantique, contextuelle).

Étape 3 : Exécution de la branche choisie (Neo4j ou FAISS).

Étape 4 : L'agent évalue la qualité de la réponse (reward) et met à jour la Q-table.

Étape 5 : Affichage du résultat enrichi et du raisonnement (policy path + reward).

4-Bouton Query



Élément	Fonction
Champ de saisie principal	Zone pour écrire la question librement. C'est ici que l'utilisateur formule sa requête.
Bouton «Envoyer / Exécuter»	Lance le traitement de la query vers le module Q-Learning pour classification (sémantique ou systématique).
Analyse automatique de la requête	Affiche en temps réel le type détecté : <i>Semantic</i> , <i>Systematic</i> , ou <i>Hybrid</i> .
Score de confiance	Indique la probabilité du routage choisi (ex. : "Graph RAG – Confiance 92 %").
Prévisualisation du routage	Montre visuellement vers quelle branche la requête sera dirigée (Graph RAG ou Vectoriel RAG).
Historique des requêtes	Liste des dernières questions posées, utile pour tester la cohérence du modèle.
Bouton «Réinitialiser»	Vide le champ et remet le système à zéro pour une nouvelle requête.

Liaison entre Frontend et Backend

- Frontend- **Frontend (HTML/React)** → crée les boutons et l'interface.
- **Backend (FastAPI)** → fournit les endpoints que les boutons appellent.

Le Frontend Rest au choix , soit le HTML soit React [Java]

En **FastAPI**, tu ne crées pas directement les *boutons* de ton application — les boutons appartiennent au **frontend React**.

Le rôle de FastAPI est de fournir des **endpoints REST** que tes boutons vont appeler. Par exemple :

- **Bouton Query (React)** → appelle `POST /query` sur FastAPI.
- **Bouton Vectoriel RAG (React)** → Appelle `GET /vectorial` pour lancer chunking, embeddings FAISS, retrieval.
- **Bouton Graph RAG (React)** → appelle `GET /graph` pour afficher graphe Aura, communautés Louvain, analyse avancée.
- **Bouton Agentic RAG (React)** → appelle `GET /agentic` pour exécuter la logique Q-Learning et afficher la décision.

Organisation claire

- **Frontend (React)** Tu crées les boutons et l'interface utilisateur. Chaque bouton déclenche une requête HTTP vers ton backend.
- **Backend (FastAPI)** Tu définis les routes (endpoints).

```
from fastapi import FastAPI

app = FastAPI()

@app.post("/query")
def process_query(query: str):
    # Analyse automatique + routage Q-Learning
    return {"decision": "Graph RAG", "confidence": 0.92}

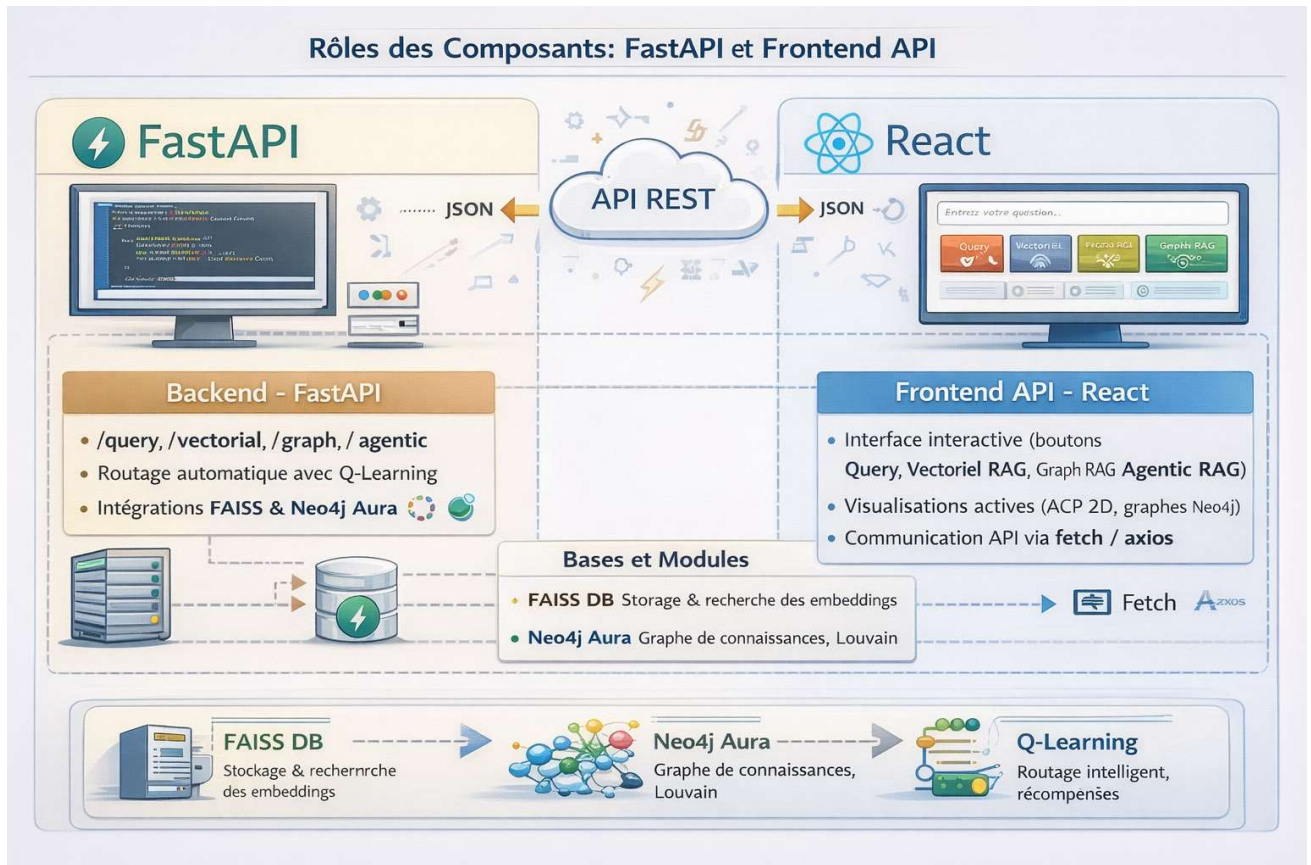
@app.get("/vectorial")
def vectorial_rag():
    # Chunking + embeddings FAISS + retrieval
    return {"best_chunking": "Recursive Splitting"}

@app.get("/graph")
def graph_rag():
    # Graphe Aura + communautés Louvain
    return {"clusters": 4, "modularity": 0.82}

@app.get("/agentic")
def agentic_rag():
    # Q-Learning policy + reward
    return {"policy": "Graph RAG", "reward": 1}
```

En résumé :

- React Ou HTML [au choix]= boutons + interface
- FastAPI = logique + endpoints
- Les deux communiquent via API REST.



2-Inforgraphie de FASTAPI



FastAPI

API moderne, rapide (haute performance) pour Python
Basé sur les standards ouverts de l'ASGI, avec validation des données par Pydantic.

Rapide • Moderne • Facile

Caractéristiques clés

-  **Ultra Rapide**
Performance proche de NodeJS et Go
-  **Validation Automatique**
Grâce à Pydantic, avec sérialisation et désérialisation
-  **Documentation Interactive**
Swagger UI et ReDoc intégrés automatiquement
-  **Basé sur les Standards**
OpenAPI (Swagger) et ASGI
-  **Facile à Utiliser**
Moins de code, plus de productivité

Exemple de code

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI(
    title="Mon API avec FastAPI",
    version="1.0.0"
)

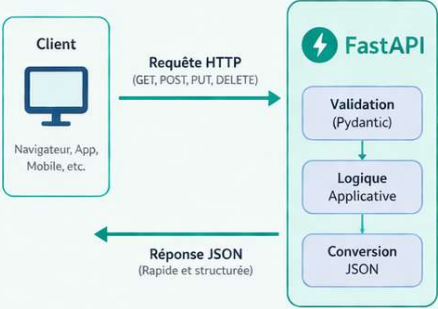
class Item(BaseModel):
    nom: str
    prix: float
    disponible: bool = True

@app.get("/")
def accueil():
    return {"message": "Bienvenue sur FastAPI !"}

@app.post("/items/")
def creer_item(item: Item):
    return {"item": item, "statut": "créé avec succès"}
```

Python 3.8+ • Moins de code, plus d'efficacité

Comment ça fonctionne ?



Documentation automatique
Dès que vous lancez votre application, deux interfaces de documentation sont disponibles :

[/docs](#)
(Swagger UI)

[/redoc](#)
(ReDoc)

Pour tester et explorer facilement votre API.

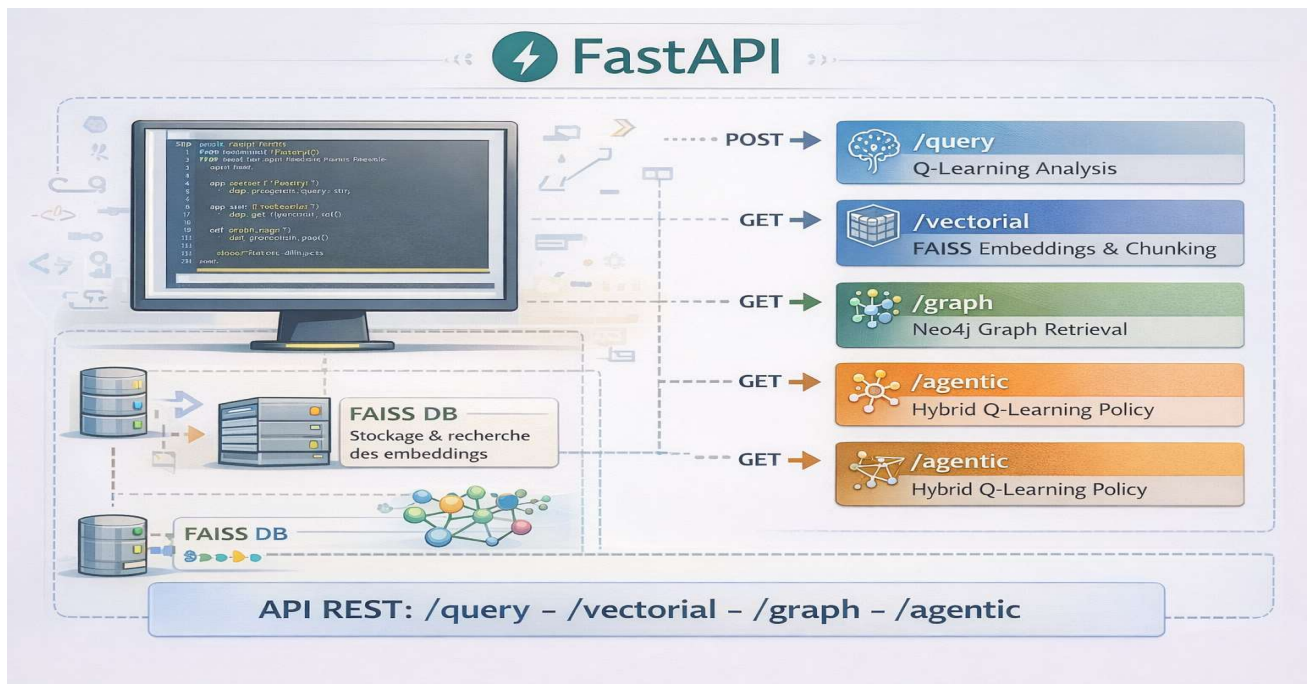
 HAUTE PERFORMANCE

 100% PYTHON

 OPEN SOURCE

 PRODUCTION READY

 COMMUNAUTÉ ACTIVE



Tutorial pour facilement Travailler Fast-API

<https://www.datacamp.com/tutorial/introduction-fastapi-tutorial>